MIT 8103 ADVANCED DATABASE SYSTEMS

University/Student Result Portal

Technical Architecture README & Portfolio Evaluation Summary

Author: Samuel Chukwuma
Student ID: 301910606
Engines: PostgreSQL & MongoDB
Source Repository: https://github.com/Weltraum-Fahrer/Chukwuma Samuel 301910606

01 System Overview & Operational Scope

The University/Student Result Portal is a hybrid relational and document-oriented database system managing academic assessment records with strict data integrity and auditable event histories. It serves students and academic staff across three core workflows:

- **Result Recording:** Persist individual student scores against course offerings with automatic timestamps.
- **Targeted Retrieval:** Audit recorded results filtered by student identifier, course code, and term.
- **Analytical Aggregation:** Compute result counts and mean scores across cohorts.

02 Relational Schema & Integrity Modelling (Portfolio 1)

The transactional core is normalized to 3NF across four decoupled tables— `students`, `courses`, `offerings`, and `results`. Primary and foreign keys enforce uniqueness and referential integrity. Domain CHECK constraints enforce invariants (e.g., `starts_on <= ends_on`). During testing, all 13 deliberately invalid inserts were cleanly rejected.

Figure 1. Relational Entity-Relationship Schema Table.

Table	Key Constraints	Domain Role & Enforced Invariants
students	PK: `student_id`	Stores enrollment dates and account status.
courses	PK: `course_id` UK: `course_code`	Defines catalog courses with unique codes and titles.
offerings	PK: `offering_id` FK: `course_id`	Binds courses to terms; enforces `starts_on <= ends_on`.
results	PK: `result_id` FK: `student_id`, `offering_id`	Records scores and audit timestamps; invalid inserts blocked.

03 Query Processing & Index Optimisation (Portfolio 2)

Three representative SQL queries validated multi-table joins. Benchmarking a dedicated dataset of 20,000 rows showed the targeted index reduced median lookup latency from 0.944 ms to 0.181 ms. EXPLAIN plans confirmed direct index utilization.

Evaluation Phase	Dataset Scale	Observed Metric	Verification Outcome
Functional SQL Suite	Sample records	2 rows returned	Confirmed 2026FA paired with SYN101
Baseline Lookup	20,000 rows	0.944 ms latency	Baseline reference captured
Indexed Lookup	20,000 rows	0.181 ms latency	Identical rows; EXPLAIN confirmed index scan

04 Transaction Isolation & Concurrency Control (Portfolio 3)

ACID isolation was verified using a transient counter. Under READ COMMITTED isolation, uncommitted modifications remained invisible to concurrent sessions. Concurrent read-modify-write contention without locks suffered a lost update (final value: 2), while pessimistic row locking (`SELECT FOR UPDATE`) properly serialized execution (final value: 3).

Concurrency Scenario	Execution Behavior	Outcome / Final State
READ COMMITTED & ROLLBACK	Session A modified counter without commit; Session B queried concurrently.	Dirty read prevented; ROLLBACK restored prior value.
Concurrent Update (Unlocked)	Sessions read the initial counter concurrently and applied separate increments.	Lost-update anomaly; final counter left at 2.
Pessimistic Locking	Session A locked row before read; Session B waited for Session A's COMMIT.	Serialized execution; final counter reached 3.

05 Polyglot Persistence & NoSQL Event Modelling (Portfolio 4)

Domain lifecycle events were modeled in MongoDB using a `result_events` collection. Schema validation enforced structural rules by rejecting invalid dates and duplicate IDs. Documents pair embedded entity snapshots for single-read efficiency with a polymorphic details sub-document, while PostgreSQL remains the relational system of record.

06 Distributed Topology & Cloud Availability (Portfolio 5)

For high availability of the `result_events` workload, a three-member MongoDB replica set distributed across separate failure zones was specified. The architecture leverages automated primary elections and majority write acknowledgements for fault tolerance, separate from PostgreSQL's infrastructure.

07 Principal Engineering Reflection

- **Storage-Boundary Invariants:** Normalization and declarative constraints prevent invalid state from ever entering persistent storage.
- **Serializing Read-Modify-Write Paths:** Acquiring explicit row locks prior to reading enforces strict write serialization and prevents silent overwrites.
- **Execution Plans:** Inspecting EXPLAIN plans is mandatory to validate indexing decisions over simple output equivalence.

08 Attestation

I independently designed and verified the database architecture, schema, constraints, and tests. While GitHub Copilot and the portfolio generator assisted with drafting boilerplate scripts and test execution, I personally audited all outputs, validated query plans, and take full responsibility for the submitted work.

Samuel Chukwuma | Student ID: 301910606 | MIT 8103 Advanced Database Systems

09 Repository Evidence & Technical References

SQL scripts, MongoDB code, EXPLAIN plans, and outputs are archived at https://github.com/Weltraum-Fahrer/Chukwuma Samuel 301910606.

Technology Domain	Canonical Documentation & Standards Referenced
PostgreSQL	Constraints (PK, FK, UNIQUE, CHECK); Execution Plans (EXPLAIN); Isolation (READ COMMITTED); Locks (SELECT FOR UPDATE).
MongoDB	Data Modelling; Replica Sets & Elections; Horizontal Sharding.
Diagramming	Mermaid Entity-Relationship (ER) Diagrams.